

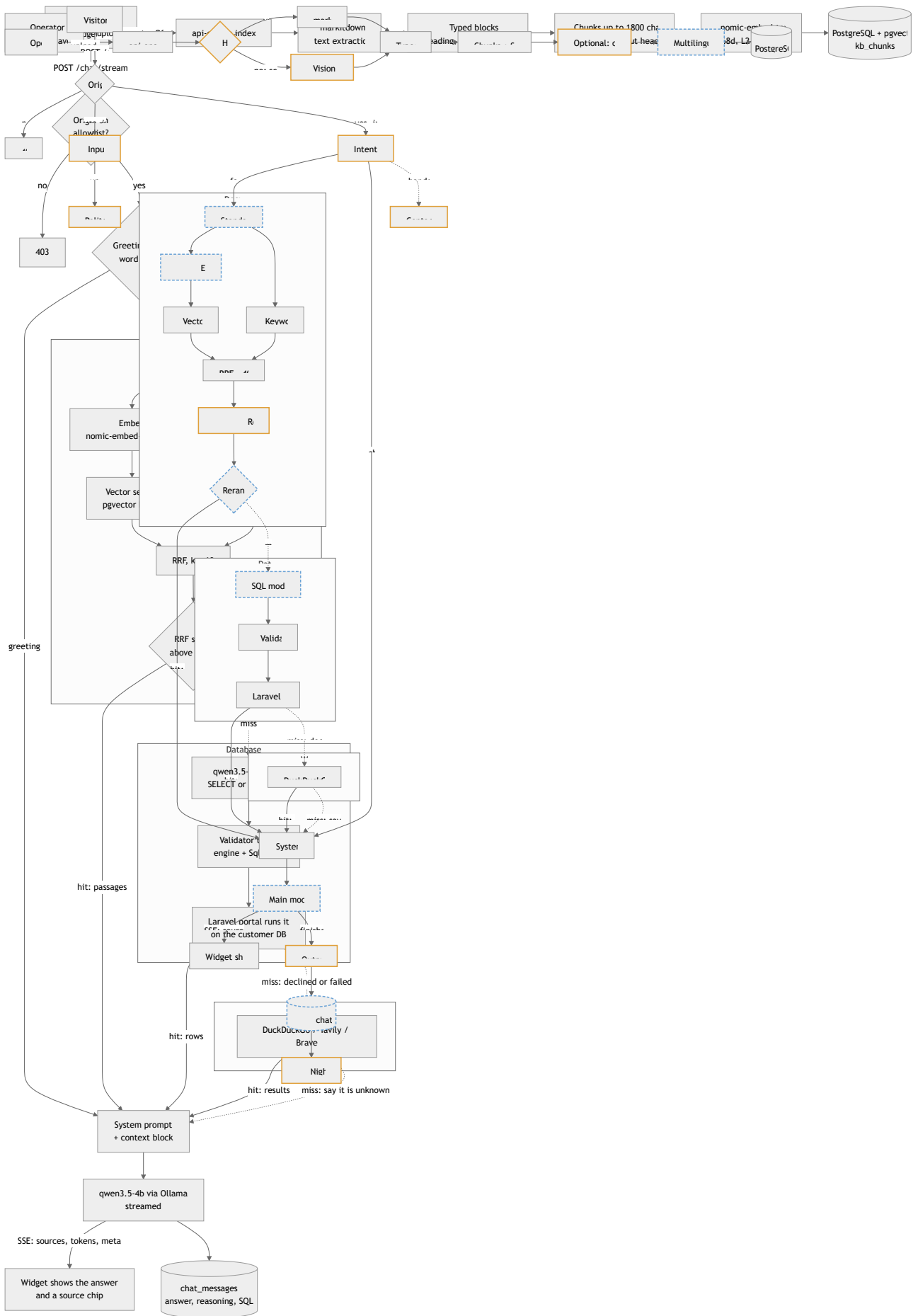
# DGX Spark Model Stack

Today one 4B model answers, writes SQL and would route if anything did. This page draws that pipeline as the code has it, then the pipeline planned for two DGX Sparks, and marks every stage that changes.

|                                                                                                                          |                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Follow-ups search the wrong text</b><br>Sources get the raw message only. "and the warranty?" is what gets embedded.  | <b>The relevance floor is a guess</b><br>It sits on an RRF rank sum, and two migrations have re-tuned it already. |
| <b>A reranker is the cheapest big gain</b><br>About 1–8 GB, and it gives the floor a score that means "this answers it". | <b>Intent should pick the reply, not the source</b><br>Your source order stays in charge of KB vs DB vs web.      |

● TODAY

## The chat path, as the code runs it



One model, gwen3.5-4b, writes both the SQL and the answer. The only other model is nomic-embed-text. The documents → database → web order shown is one example; each bot sets its own.

Ingestion. markdown reads the text layer, so a scanned PDF indexes as nothing.

Already right, and kept

- Hybrid dense + keyword retrieval with rank fusion
- Structure-aware chunks with Section / About headers
- `embedding_model` on every chunk, so vector spaces never mix
- An operator-set source order, so answers can be explained
- SQL validated twice, against a read-only account
- No source failure ends a conversation

Missing, by cost to answer quality

1. Follow-ups: history never reaches retrieval or SQL — routers/chat.py passes req.message
2. The floor sits on an uncalibrated RRF score
3. No reranker — the old objection was PyTorch in the engine
4. English-centric embedding for Malay and mixed questions
5. Scanned documents yield no text
6. No safety model on input or output
7. Nothing measures whether a model change helped

● PROPOSED

The chat path on two Sparks

☐ New stage    ☒ Exists today, changes    ☐ Unchanged

The cascade block is the same as today. What's new sits around it: a front stage that decides what kind of reply this is and rewrites the question, a reranker inside Documents, and a guard plus measurement after the answer. Image generation is deferred and not drawn.



Ingestion. Truncate to 1024 dimensions because pgvector's HNSW index takes at most 2,000; Qwen3-Embedding-8B produces 4,096 natively.

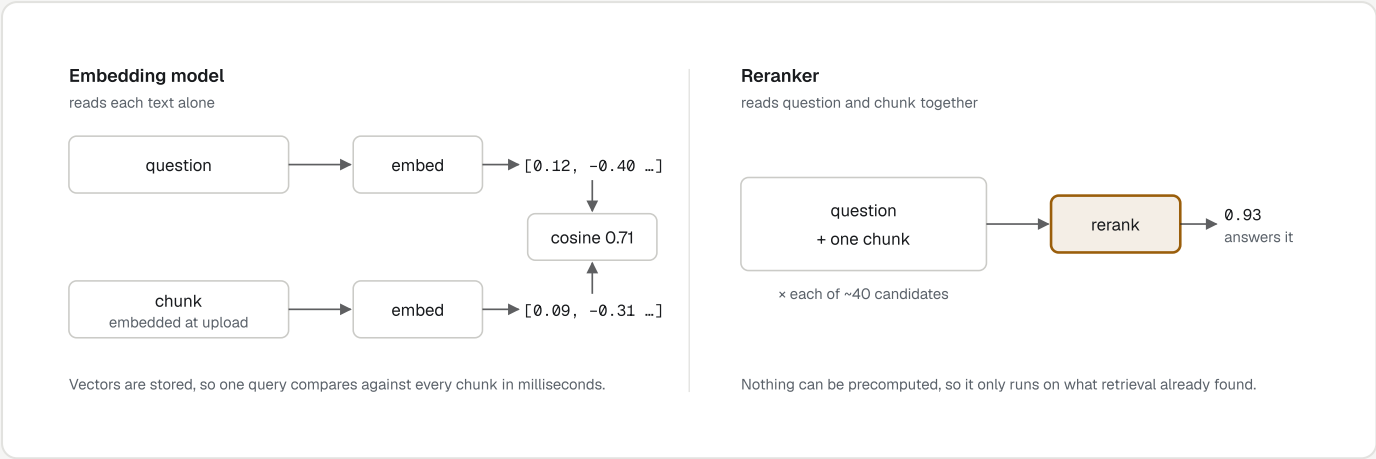
STAGE BY STAGE

## What changes

| STAGE           |         | TODAY                   | PROPOSED                              | WHY                                       |
|-----------------|---------|-------------------------|---------------------------------------|-------------------------------------------|
| Follow-ups      | added   | Raw message searched    | Standalone query from the intent call | Fixes "and the warranty?"                 |
| Candidates      | added   | Top k by RRF            | ~40 by RRF, reranked to 5             | Largest retrieval gain per GB             |
| Relevance floor | changed | RRF score               | Reranker score                        | Makes the cascade's hit / miss honest     |
| Answer model    | changed | qwen3.5-4b, Ollama      | ~120B-class MoE, NVFP4, vLLM          | The main quality jump                     |
| Embedding       | changed | nomic-embed-text, 768d  | Multilingual, 1024d                   | Malay and mixed-language queries          |
| SQL             | changed | The chat model          | Coder MoE, same validators            | Joins and dialects                        |
| Greeting gate   | changed | Word rules              | Intent call; rules kept as fast path  | Tells chat, facts and image apart path    |
| Safety          | added   | Origin + SQL rules      | Input guard, output guard             | One tenant's bot saying something harmful |
| Upload parsing  | added   | Text layer only         | Vision OCR when there is none         | Scans index as nothing today              |
| Chunk context   | changed | Section / About headers | Kept, plus optional model sentence    | Was blocked only by model size            |
| Logging         | changed | Answer, reasoning, SQL  | + intent, scores, model per role      | "Why did it say that"                     |
| Evaluation      | added   | None                    | Golden set, nightly judge             | A model swap becomes measurable           |
| Source choice   | kept    | Operator order          | Operator order                        | Keeps answers explainable                 |
| Image           | added   | None                    | Separate diffusion service            | New capability, built last                |

WHY THEY'RE TWO MODELS

# Embedding finds candidates. Reranking judges them.



They look alike because both read text and return a number. The embedding model matches topic across the whole corpus; the reranker checks whether a passage actually answers the question. Its score is what the relevance floor should use.

## THE OPEN QUESTION

# Where the intent model fits

You removed a model router on 14 September, and not because it routed badly. Your spec says it plainly: an operator couldn't configure it, predict it, or explain an answer that came from the wrong place. An intent model that picks between knowledge base and database brings that router back under another name.

**Recommendation:** intent decides *what kind of reply* this is. The bot's source order still decides *which source* supplies facts.

| VERDICT | GOES TO                                                   | OPERATOR CONTROL               |
|---------|-----------------------------------------------------------|--------------------------------|
| chat    | Main model, no source consulted                           | None needed                    |
| facts   | The cascade, in the bot's order, with the rewritten query | Source order, unchanged        |
| image   | Image service                                             | Per-bot switch, off by default |
| handoff | Contact details or a human, later                         | Per-bot switch                 |

## One call, two jobs

The rewrite rides on the intent call, so fixing follow-ups adds no extra call.

```
{
  "intent": "facts",
  "query": "What is the warranty perio
  "language": "ms"
}
```

## Rules that keep it predictable

- 1 Structured output at temperature 0. vLLM holds the reply to the JSON schema, so it can't narrate, and the same message gets the same verdict.
- 2 Lean toward looking things up. If the word rules call it a question and intent says chat , treat it as facts .
- 3 Log the verdict on the message and show it in the logs screen.

| STEP                            | ADDS BEFORE FIRST TOKEN | WHEN                                   |
|---------------------------------|-------------------------|----------------------------------------|
| Input guard, 4B                 | 0.1-0.3 s               | Every message, in parallel with intent |
| Intent + rewrite, 3B-active MoE | 0.3-0.8 s               | Every message                          |
| Rerank ~40 candidates           | 0.2-0.5 s               | When documents take a turn             |
| SQL generation                  | 0.5-1.5 s               | When the database takes a turn         |

Latency figures are estimates. Replace them with measurements on the real hardware.

### MODEL ROSTER

## Nine jobs, six deployments

9 jobs that need a model      6 models deployed on the Sparks

3 models on the 8 GB machine now

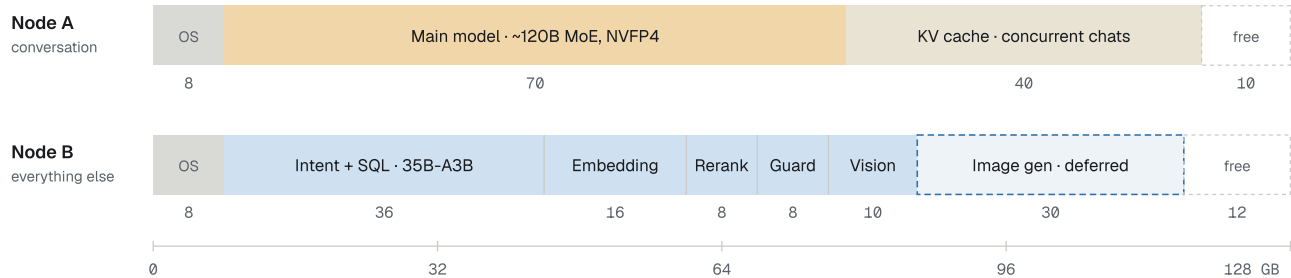
Intent and SQL share one model, the answer model also judges evaluations, and one guard model checks both directions at first. A role left blank falls back to the bot's own model. The reranker and vision are the exception: they have no stand-in, so their stage is skipped instead.

| JOB              | SETTING                   | ON THE SPARKS                             | ALTERNATIVE                                  | 8 GB MACHINE NOW                                         | SERVED BY                                                            |
|------------------|---------------------------|-------------------------------------------|----------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| Answer           | bot's AI provider         | Qwen3.5-122B-A10B, NVFP4                  | gpt-oss-120b                                 | qwen3.5-4b                                               | vLLM, node A                                                         |
| Embedding        | embedding_*               | Qwen3-Embedding-4B, 1024d                 | bge-m3                                       | nomic-embed-text                                         | vLLM<br><code>/v1/embeddings</code>                                  |
| Reranker         | rerank_model_*            | Qwen3-Reranker-4B                         | bge-reranker-v2-m3                           | bge-reranker-v2-m3<br><code>no stand-in</code>           | vLLM <code>/v1/rerank</code> ,<br>llama.cpp <code>--reranking</code> |
| Intent + rewrite | intent_model_*            | Qwen3.5-35B-A3B                           | fine-tuned Qwen3.5-4B                        | qwen3.5-4b                                               | vLLM, node B, shared with SQL                                        |
| SQL              | sql_model_*               | Qwen3-Coder-30B-A3B, or share the 35B-A3B | —                                            | qwen3.5-4b                                               | vLLM, node B                                                         |
| Input guard      | guard_model_*             | Qwen3Guard-Gen-4B                         | Llama Guard 4 12B                            | Qwen3Guard-Gen-0.6B, or qwen3.5-4b with a prompt         | vLLM, node B                                                         |
| Output guard     | guard_model_*             | Qwen3Guard-Gen-4B, finished answer        | Qwen3Guard-Stream-4B, checks while streaming | same as input guard                                      | vLLM, node B                                                         |
| Vision OCR       | vision_model_*<br>· later | Qwen3-VL-8B                               | dots.ocr, olmOCR-2                           | none; scans stay unsupported<br><code>no stand-in</code> | vLLM, node B                                                         |
| Judge, for evals | judge_model_*<br>· later  | the answer model, overnight               | a hosted API                                 | a hosted API; a 4B judges poorly                         | reuses node A                                                        |
| Image            | <code>deferred</code>     | FLUX.1-dev or Qwen-Image                  | —                                            | —                                                        | diffusers or ComfyUI                                                 |

HARDWARE

# Two Sparks as two servers

Each node has 128 GB of unified memory at about 273 GB/s, so decode speed tracks *active* parameters. Nothing planned needs more than one node, and splitting a model over the 200GbE link adds a network hop to every token. Run two independent vLLM hosts instead.



Drawn to scale, in GB. On unified memory, `--gpu-memory-utilization` is a fraction of all 128 GB, so give each vLLM process on a node an explicit share and keep the shares below 1 in total.

Candidates as of September 2026, to re-check before committing: Qwen3.5-122B-A10B (main) · Qwen3.5-35B-A3B or Qwen3-Coder-30B-A3B (intent + SQL) · Qwen3-Embedding-4B / 8B · Qwen3-Reranker-4B or bge-reranker-v2-m3 · Qwen3Guard-Gen-4B.

SEQUENCE

# Order of work

## Now, on the 8 GB machine

1. Write the golden set: 50–100 questions per test bot, each with its expected source and answer, plus database questions with expected rows.
2. Log what decided each answer: the model per role and the retrieval scores.
3. Add a rerank stage behind `rerank_model_*` settings, the way `sql_model_*` works. bge-reranker-v2-m3 (~0.6B) runs today on llama.cpp `--reranking` or TEI.
4. Move the relevance floor onto the reranker score.
5. Build the intent + rewrite JSON contract against the 4B model. The contract will outlive the model.

## When the Sparks arrive

6. Run one vLLM process per role, then test two or three candidates for each role against the golden set.
7. Switch the embedding model, truncate to 1024d and rebuild the index.
8. Add the input guard, then the output guard on finished answers.
9. Add vision OCR for scanned uploads. Image generation is deferred.

Recorded in the repo at `docs/model-stack-review.md`.